Distributed Optimization Algorithm with Superlinear Convergence Rate

Yeming Xu, Ziyuan Guo, Kaihong Lu, Huanshui Zhang*

arXiv:2409.12392v3 [math.OC] 17 Aug 2025

Abstract

This paper considers distributed optimization problems, where each agent cooperatively minimizes the sum of local objective functions through the communication with its neighbors. The widely adopted distributed gradient method in solving this problem suffers from slow convergence rates, which motivates us to incorporate the second-order information of the objective functions. However, the challenge arises from the unique structure of the inverse of the Hessian matrix, which prevents the direct distributed implementation of the second-order method. We overcome this challenge by proposing a novel optimization framework. The key idea is to transform the distributed optimization problem into an optimal control problem. Using Pontryagin's maximum principle and the associated forward-backward difference equations (FBDEs), we derive a new distributed optimization algorithm that incorporates the second-order information without requiring the computation of the inverse of the Hessian matrix. Furthermore, the superlinear convergence of the proposed algorithm is proved under some mild assumptions. Finally, we also propose a variant of the algorithm to balance the number of iterations and communication.

I. INTRODUCTION

Consider that n agents in a connected network collaboratively solve a consensus optimization problem

$$\min_{y\in\mathbb{R}^p} f(y) \triangleq \sum_{i=1}^{n} f_i(y), \tag{1}$$

where $f : \mathbb{R}^p \mapsto \mathbb{R}$ is the objective function, while each agent i privately holds a local objective function $f_i : \mathbb{R}^p \mapsto \mathbb{R}$ and can only communicate with its neighbors. Problem (1) has received extensive attention in recent years due to its wide applications in many areas such as sensor networks [1], [2], [3], distributed control [4], [5], [6], and machine learning [7], [8], [9], etc.

Second-order methods are attractive because of their faster convergence rates, making it meaningful to consider their distributed implementations. Unfortunately, since the inverse of the Hessian matrix is dense, it is rather difficult to apply second-order methods directly to handling distributed optimization problems. In fact, some Newton-type methods have been proposed to address this issue and have achieved notable progress. The distributed Adaptive Newton (DAN) algorithm with a global quadratic convergence rate is proposed in [10], but this algorithm requires running a finite-time consensus inner loop in each iteration. Moreover, in a master-slave network configuration, some distributed quasi-Newton methods were proposed in [11], [12], [13], but the master node in this setup uses all the information from the slave nodes. The network Newton (NN) method approximates the Newton step by truncating the Taylor expansion of the inverse Hessian matrix [14]. Another method, known as distributed quasi-Newton (DQN), represents a different family of distributed Newton-like methods to solve problem (1) [15]. Using the penalty method, DQN and NN converge to a solution neighborhood. To further improve the convergence rate and solution accuracy, second-order methods working in the primal-dual domain have been proposed [16], [17], [18]. The Newton tracking algorithm updates the local variable using a local Newton direction modified by neighboring and historical information [19]. The distributed Newton-Raphson

H. Zhang is with the College of Electrical Engineering and Automation, Shandong University of Science and Technology, Qingdao 266590, China, and also with the School of Control Science and Engineering, Shandong University, Jinan, Shandong 250061, China (e-mail: hszhang@sdu.edu.cn).

Y. Xu, Z. Guo, and K. Lu are with the School of Electrical and Automation Engineering, Shandong University of Science and Technology, Qingdao 266590, China (e-mail: ymxu2022@163.com; skdgzy@sdust.edu.cn; khong_lu@163.com).

This work was supported by the Original Exploratory Program Project of the National Natural Science Foundation of China (62450004), the Joint Funds of the National Natural Science Foundation of China (U23A20325), and the Major Basic Research of Natural Science Foundation of Shandong Province (ZR2021ZD14). (Corresponding author: Huanshui Zhang).

This work has been submitted to the IEEE for possible publication. Copyright may be transferred without notice, after which this version may no longer be accessible.

method proposed in [20] requires exchanging gradient information and Hessian matrix information to estimate the global Newton direction. Both the Newton tracking and Newton-Raphson algorithms converge linearly to the optimal solution. Although these methods successfully incorporate second-order information into distributed optimization, they primarily focus on directly modifying or approximating Newton step.

In our recent work, we successfully applied optimal control to solve optimization problems and achieved a series of significant results [21], [22], [23]. Unlike the traditional Newton-type methods, this paper proposes a new second-order method based on optimal control theory. We first reformulate the problem using a penalty method to exploit its specific structure, and then transform the optimization problem into an optimal control problem where the update size of each iteration is the control input, and the control objective is to design the current control input to minimize the sum of the original objective function and the update size for the future time instant. Using Pontryagin's maximum principle, we derive a new optimization algorithm. The proposed algorithm uses only neighbor information and incorporates second-order information through a special optimization structure. It avoids using the inverse of the Hessian matrix, significantly reducing computational complexity. Furthermore, we prove the superlinear convergence of the algorithm and establish its relationship with traditional algorithms. To balance the number of iterations and communication, we also present a variant of the algorithm. It is worth noting that with appropriate parameter selection, this variant can also inherit the aforementioned advantages.

The remainder is arranged as follows. In Section II, the distributed optimization problem is converted into an optimal control problem. In Section III, the novel algorithms, along with the variant to balance the number of iterations and communication, are proposed. The convergence analysis of these proposed algorithms is in Section IV. Concluding remarks of Section V complete the paper.

Notation: We use $\|A\|$ and $\|x\|$ to denote the Euclidean norm of matrix $A \in \mathbb{R}^{n\times n}$ and vector $x \in \mathbb{R}^n$, respectively; $\mathbb{R}^n$ is the set of n-dimensional real vectors; $\mathbb{S}^n_{++}$ denotes the set of real symmetric positive definite matrices; We use I and 0 to denote the identity matrix and all-zero matrix of proper dimensions; For symmetric matrices A and B, $A \succ B$ means that the matrix $A - B$ is positive semidefinite. Denote $\nabla f(x)$ and $\nabla^2 f(x)$ as the gradient and the Hessian matrix of $f(x)$; The eigenvalues $\lambda_i(A)$ of a matrix $A \in \mathbb{R}^{n\times n}$ are indexed in an decreasing order with respect to their real parts, i.e., $\Re(\lambda_1(A)) \geq \cdots \geq \Re(\lambda_n(A))$.

II. PROBLEM FORMULATION

The communication network among nodes is modeled by an undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{1, \ldots, n\}$ denotes the set of nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. An edge $(i, j) \in \mathcal{E}$ indicates that nodes i and j can directly send messages to each other. The neighbors of node i are denoted by $\mathcal{N}_i$. The weight matrix of $\mathcal{G}$ is a symmetric, doubly stochastic $n \times n$ matrix W with elements $w_{ij} \geq 0$. The weight $w_{ij} = 0$ if and only if $j \notin \mathcal{N}_i$, and $w_{ii} = 1 - \sum_{j\in\mathcal{N}_i} w_{ij} > 0$.

We first provide a penalty interpretation of problem (1) to further explain the decentralized gradient descent algorithm. Then, we introduce an innovative idea for handling this problem using optimal control theory.

As we can see from [14], [15], auxiliary penalty function $F : \mathbb{R}^{np} \mapsto \mathbb{R}$ of problem (1) can be written as

$$\min_{x\in\mathbb{R}^{np}} F(x) \triangleq h(x) + \frac{1}{2\lambda} x^T (I - Z)x, \tag{2}$$

where $x = (x^1, \ldots, x^n) \in \mathbb{R}^{np}$, $h(x) = \sum_{i=1}^{n} f_i(x^i)$, $x^i \in \mathbb{R}^p$ is the local decision vector of agent i, λ is the penalty coefficient, and $Z = W \otimes I \in \mathbb{R}^{np\times np}$ is the Kronecker product of W and I. A common numerical method to solve problem (2) is using the gradient descent

$$x_{k+1} = x_k - \lambda \nabla F(x_k), \tag{3}$$

where $\nabla F(x_k) = \nabla h(x_k) + \frac{1}{\lambda}(I - Z)x_k$ and $\nabla h(x_k) = (\nabla f_1(x_k^1), \ldots, \nabla f_n(x_k^n))$. Indeed, the decentralized gradient descent can be considered as the component form of (3), i.e., the ith component of (3) is

$$x_{k+1}^i = \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij} x_k^j - \lambda \nabla f_i(x_k^j), i = 1, \ldots, n. \tag{4}$$

Although the smaller step size means the solution of problem (2) is closer to the solution of problem (1), it also leads to slower convergence.

Gradient descent (3) can be framed as the update of the state of a discrete-time linear time-invariant system

$$x_{k+1} = x_k + u_k, \tag{5}$$

where $x_k \in \mathbb{R}^{np}$ and $u_k = -\lambda \nabla F(x_k) \in \mathbb{R}^{np}$ are the state and control, respectively. Inspired by this, we aim to propose a numerical method to solve the problem (2) by finding a new control law u_k to update the state x_k within the framework of the optimal control problem, i.e.,

$$\begin{aligned} &\min_{u} \sum_{k=0}^{N} (F(x_k) + \tfrac{1}{2} u_k^{\mathrm{T}} R u_k) + F(x_{N+1}), \\ &\text{subject to (5)}, \end{aligned} \tag{6}$$

where the initial condition x_0 is given, N is the time horizon. The terminal cost is $F(x_{N+1})$ and $R \in \mathbb{S}_{++}^{np}$ is the control weighted matrix.

Remark 1. *It is apparent from* (6) *that we minimize the accumulation of $F(x_k)$ and $u_k^{\mathrm{T}} R u_k$. The optimal solution u_t is in charge of regulating x_{t+1} to minimize $\sum_{k=t+1}^{N} [F(x_k) + \frac{1}{2} u_k^T R u_k] + F(x_{N+1})$. This means that we aim to control x_k to reach the minimum point of $F(x)$ as quickly as possible while avoiding excessive costs associated with large controls. The purpose of transforming problem* (2) *into problem* (6) *is to theoretically derive the fastest and stable algorithm with the form of* (5)*.*

III. DISTRIBUTED OPTIMIZATION METHOD VIA OPTIMAL CONTROL

In this section, we use optimal control theory to solve problem (6). This inspires us to develop an optimization algorithm that can be implemented in a distributed manner. Additionally, a variant of the algorithm is also proposed to balance the number of iterations and communication.

A. Algorithm Development

Following from [24], applying Pontryagin's maximum principle to problem (6) to yield the following costate equations:

$$p_{k-1} = p_k + \nabla F(x_k), \tag{7}$$

$$0 = R u_k + p_k, \tag{8}$$

with the terminal condition $p_N = \nabla F(x_{N+1})$. From the FBDEs (5), (7), (8), we can derive the relationship between the optimal control law u_k and the optimal state trajectory x_k:

$$u_k = -R^{-1} \sum_{t=k+1}^{N+1} \nabla F(x_t), \tag{9}$$

$$x_{k+1} = x_k - R^{-1} \sum_{t=k+1}^{N+1} \nabla F(x_t). \tag{10}$$

Noted that (10) does not apply to solving the optimal state directly due to its noncausality. However, the special nonlinear form of (10) can be implemented by solving for (x_k, u_k, p_k) such that the FBDEs (5), (7), (8) are satisfied with the aid of the method of successive approximations (MSA) [21]. A straightforward algorithm will be presented in this subsection to simplify the calculation. We define the first-order Taylor expansion of (9) at x_{t-1} as

$$g_k \triangleq -R^{-1} \sum_{t=k+1}^{N+1} \big[\nabla h(x_{t-1}) + \nabla^2 h(x_{t-1})(x_t - x_{t-1}) + \frac{1}{\lambda}(I - Z) x_t\big], \tag{11}$$

where $\nabla^2 h(x)$ is the block diagonal matrix with the ith diagonal block $\nabla^2 f_i(x^i)$. Then, an approximate recursion for (10) is provided in Lemma 1.

Lemma 1. *Dynamics* (10) *is approximated by the following dynamics*

$$x_{k+1} = x_k + g_k, \tag{12}$$

$$g_k = -\big[R + \nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z)\big]^{-1}\big[\nabla h(x_k) + \frac{1}{\lambda}(I - Z)x_k + \sum_{t=k+1}^{N}(\nabla h(x_t) + \frac{1}{\lambda}(I - Z)x_t - (\nabla^2 h(x_t) + \frac{1}{\lambda}(I - Z))g_t)\big], \tag{13}$$

$$g_N = -\big[R + \nabla^2 h(x_N) + \frac{1}{\lambda}(I - Z)\big]^{-1}\big[\nabla h(x_N) + \frac{1}{\lambda}(I - Z)x_N\big]. \tag{14}$$

Proof. The conclusion is achieved by substituting the first-order Taylor expansion g_k of the optimal control u_k into (5), refer to [22] for the details. □

Remark 2. *Let* $N = k$, (12)-(14) *can be recovered as the centralized Newton method when* $R = 0$.

From lemma 1, we say that x_k generated by algorithm (12)-(14) is almost along the optimal state trajectory (10). Note that (12)-(14) still exhibit noncausality. To further eliminate the remaining noncausality and develop a distributed optimization algorithm, we modify (12)-(14) in a similar way as [23], which obtains the following distributed optimization algorithm based on optimal control (DOBOC):

$$x_{k+1} = x_k - \hat{g}_k(x_k), \tag{15}$$

$$\hat{g}_l(x_k) = \eta\big[\nabla h(x_k) + \frac{1}{\lambda}(I - Z)x_k\big] + \big[I - \eta(\nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z))\big]\hat{g}_{l-1}(x_k), \tag{16}$$

$$\hat{g}_0(x_k) = \eta\big[\nabla h(x_k) + \frac{1}{\lambda}(I - Z)x_k\big], l = 1, \ldots, k, \tag{17}$$

where $\eta > 0$ is an adjustable scalar. The key reason we replace $\big[R + \nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z)\big]^{-1}$ with the scalar η in (12)-(14) is that $\big[R + \nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z)\big]^{-1}$ is dense. By making this substitution, the second-order information $\nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z)$ is used in the DOBOC algorithm, which is as sparse as the network. Relying on the optimization framework based on optimal control, the DOBOC algorithm can be implemented in a distributed manner.

Lemma 2. *The distribute implementation of the DOBOC algorithm for agent* i *is*

$$x_{k+1}^i = x_k^i - \hat{g}_k^i, \tag{18}$$

$$\hat{g}_l^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j \in \mathcal{N}_i \cup \{i\}} w_{ij} x_k^j)\big] + (1 - \frac{\eta}{\lambda})\hat{g}_{l-1}^i - \eta\nabla^2 f_i(x_k^i)\hat{g}_{l-1}^i + \frac{\eta}{\lambda}\sum_{j \in \mathcal{N}_i \cup \{i\}} w_{ij}\hat{g}_{l-1}^j, \tag{19}$$

$$\hat{g}_0^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j \in \mathcal{N}_i \cup \{i\}} w_{ij} x_k^j)\big], l = 1, \ldots, k. \tag{20}$$

Proof. The proof is immediately completed by rewriting (15)-(17) as the component form. □

The following remark illustrates the relationship between the DOBOC and the traditional optimization method.

Remark 3. *Noting that* (16)-(17) *can be further recursively written as*

$$\hat{g_k}(x_k) = [I - (I - \eta(\nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z)))^{k+1}]\big[\nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z)\big]^{-1}\big[\nabla h(x_k) + \frac{1}{\lambda}(I - Z)x_k\big]. \tag{21}$$

It is still necessary to compute the inverse of $\nabla^2 h(x_k) + \frac{1}{\lambda}(I - Z)$ *in* (21)*, which prevents directly applying* (21) *for distributed optimization. The decentralized gradient descent* (4) *is included in our approach as a special case if we set* $\eta = \lambda$ *and* $k = 0$*. If* η *is chosen appropriately,* $\hat{g}_k(x_k)$ *can converge to the Newton step.*

Now we summarize the DOBOC algorithm from the perspective of each agent i for handling problem (1), refer to Algorithm 1 for details. This algorithm only requires communication with neighbors, as described in Steps 3 and 6. Step 4 initializes the loop by computing the initial $\hat{g}_0^i$. Steps 6 and 7, as manifestations of optimal control theory, require receiving $\hat{g}_t^j$ from neighbors to obtain the update size for Step 9.

Algorithm 1 DOBOC.

Initialization: Each node i requires $\eta, \lambda > 0$ and sets $x_0^i \in \mathbb{R}^p$.
for $k = 0, 1, \ldots$ **do**
  Each agent i transmits x_k^i to all neighbors $j \in \mathcal{N}_i$ and receives x_k^j from all $j \in \mathcal{N}_i$.
  Each agent i calculates $\hat{g}_0^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}x_k^j)\big]$.
  for $l = 0, \ldots, k-1$ **do**
    Each agent i receives $\hat{g}_l^j$ with neighbors $j \in \mathcal{N}_i$.
    Each agent i updates $\hat{g}_{l+1}^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}x_k^j)\big] + (1 - \frac{\eta}{\lambda})\hat{g}_l^i - \eta\nabla^2 f_i(x_k^i)\hat{g}_l^i + \frac{\eta}{\lambda}\sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}\hat{g}_l^j$.
  end for
  Each agent i updates $x_{k+1}^i = x_k^i - \hat{g}_k^i$.
end for

B. A Usable Variant of DOBOC

During the kth iteration, the DOBOC algorithm requires agent i to transmit $\hat{g}_t^i, t = 0, \ldots, k-1$ to its neighbors. To balance the number of iterations and communication, we propose a usable variant of the DOBOC, called the DOBOC-K algorithm, where K denotes the restricted number of communications among agents in each iteration of the DOBOC algorithm. We summerize DOBOC-K in the following lemma.

Lemma 3. *The distribute implementation of the DOBOC-K algorithm for agent i is*

$$x_{k+1}^i = x_k^i - \hat{g}_{K-1}^i, \tag{22}$$

$$\hat{g}_t^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}x_k^j)\big] + (1 - \frac{\eta}{\lambda})\hat{g}_{t-1}^i - \eta\nabla^2 f_i(x_k^i)\hat{g}_{t-1}^i + \frac{\eta}{\lambda}\sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}\hat{g}_{t-1}^j, \tag{23}$$

$$\hat{g}_0^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}x_k^j)\big], t = 0, \ldots, K-1. \tag{24}$$

If $K = 1$, i.e., agent i only needs to transmit x_k^i to its neighbors at each iteration, the DOBOC-K algorithm can degenerate into the distributed gradient descent algorithm. For $K = 2, \ldots, k$, agent i needs to additionally transmit $\hat{g}_0^i, \ldots, \hat{g}_{K-2}^i$ to its neighbors at each iteration. This means that we approximate the update size $\hat{g}_k^i$ using $\hat{g}_{K-1}^i$. The details of the distributed implementation of the DOBOC-K are summarized in Algorithm 2.

Algorithm 2 DOBOC-K.

Initialization: Each agent i requires $\eta, \lambda > 0$, K and sets $x_0^i \in \mathbb{R}^p$.
for $k = 0, 1, \ldots$ **do**
  Each agent i transmits x_k^i to all neighbors $j \in \mathcal{N}_i$ and receives x_k^j from all $j \in \mathcal{N}_i$.
  Each agent i calculates $\hat{g}_0^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}x_k^j)\big]$.
  for $t = 0, \ldots, K-2$ **do**
    Each agent i receives $\hat{g}_t^j$ with neighbors $j \in \mathcal{N}_i$.
    Each agent i updates $\hat{g}_{t+1}^i = \eta\big[\nabla f_i(x_k^i) + \frac{1}{\lambda}(x_k^i - \sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}x_k^j)\big] + (1 - \frac{\eta}{\lambda})\hat{g}_t^i - \eta\nabla^2 f_i(x_k^i)\hat{g}_t^i + \frac{\eta}{\lambda}\sum_{j\in\mathcal{N}_i\cup\{i\}} w_{ij}\hat{g}_t^j$.
  end for
  Each agent i updates $x_{k+1}^i = x_k^i - \hat{g}_{K-1}^i$.
end for

IV. CONVERGENCE ANALYSIS

In this section, we reveal the superlinear convergence rate of the DOBOC algorithm. If we limit the number of communications per iteration in the DOBOC algorithm, the convergence rate of the DOBOC algorithm will degrade to at least linear. The following assumption in this subsection is useful for our conclusion.

Assumption 1. *The local objective functions $f_i, i = 1, \ldots, n$, are twice continuously differentiable, and there exist constants $0 < m \leq M < \infty$ such that for any $x \in \mathbb{R}^p$, $mI \preceq \nabla^2 f_i(x) \preceq MI$.*

Assumption 1 is standard in the analysis of the second method, see [25]. The importance of Assumption 1 is that it allows us to establish bounds on the eigenvalues of $\nabla^2 F(x)$, as described below.

Proposition 1 ([14])**.** *Under Assumption 1, the eigenvalues of $\nabla^2 F(x)$ are uniformly bounded as $mI \preceq \nabla^2 F(x) \preceq aI$, where $a = M + \frac{2(1-w_{min})}{\lambda}$ and $w_{min} = \min_i w_{ii}$.*

According to Proposition 1, for any $x, y \in \mathbb{R}^{np}$, there holds

$$F(y) \geq F(x) + \nabla F(x)^T (y - x) + \frac{m}{2}(y - x)^T (y - x), \tag{25}$$

$$F(y) \leq F(x) + \nabla F(x)^T (y - x) + \frac{a}{2}(y - x)^T (y - x). \tag{26}$$

Let x_* be the minimum point of $F(x)$. We first prove the superlinear convergence of the DOBOC algorithm.

Theorem 1. *Under Assumption 1, if η satisfies $0 < \eta < \frac{2}{a}$ and the sequence $\{x_k\}$ generated by the DOBOC algorithm converges, then*

$$||x_{k+1} - x_*|| \leq c^{k+1} ||x_k - x_*||,$$

where $c = ||I - \eta \nabla^2 F(x_*)|| < 1$.

Proof. Direct derivation shows

$$\begin{aligned} \hat{g}_k(x_*) =& [I - (I - \eta \nabla^2 F(x_*))^{k+1}] \nabla^2 F(x_*)^{-1} \nabla F(x_*) = 0, \\ \nabla \hat{g}_k(x_*) =& I - (I - \eta \nabla^2 F(x_*))^{k+1}. \end{aligned}$$

From (15), it follows that

$$\begin{aligned} & x_{k+1} - x_* \\ =& x_k - x_* - \hat{g}_k(x_k) \\ \approx& x_k - x_* - [\hat{g}_k(x_*) + \nabla \hat{g}_k(x_*)(x_k - x_*)] \\ =& x_k - x_* - \nabla \hat{g}_k(x_*)(x_k - x_*) \\ =& (I - \nabla \hat{g}_k(x_*))(x_k - x_*) \\ =& (I - \eta \nabla^2 F(x_*))^{k+1} (x_k - x_*). \end{aligned}$$

Since $||I - \eta \nabla^2 F(x_*)|| < 1$ when $\eta < \frac{2}{a}$, it follows that $||x_{k+1} - x_*|| \leq ||I - \eta \nabla^2 F(x_*)||^{k+1} ||x_k - x_*||$, i.e., the DOBOC algorithm is superlinearly convergent. The proof is completed. □

Before analyzing the convergence of DOBOC-K, we first introduce the following customary assumption, which can be found in [10], [14], [19].

Assumption 2. *The Hessians $\nabla^2 f_i(x)$ are L-Lipschitz continuous.*

The restriction in Assumption 2 guarantees that $\nabla^2 F(x)$ is Lipschitz continuous with L, as described in the following lemma.

Lemma 4. *Consider the function $F(x)$ defined in* (2)*. If Assumptions 1 and 2 hold, then for any $x, y \in \mathbb{R}^{np}$*

$$F(y) \leq F(x) + \nabla F(x)^T (y - x) + \frac{1}{2}(y - x)^T \nabla^2 F(x)(y - x) + \frac{L}{6}||y - x||^3. \tag{27}$$

Proof. Let $y = (y_1, \ldots, y_n)$ and $x = (x_1, \ldots, x_n)$, then

$$||\nabla^2 F(y) - \nabla^2 F(x)|| = \max_{\mathrm{i}=1,\cdots\mathrm{n}} ||\nabla^2 f_i(y_i) - \nabla^2 f_i(x_i)|| \leq L \max_{\mathrm{i}} ||y_i - x_i|| \leq L||y - x||, \tag{28}$$

The subsequent proof can be obtained directly using the Taylor expansion error as discussed in [26]. □

The following lemma provides the bound for the error $F(x_{k+1}) - F(x_*)$ in terms of $F(x_k) - F(x_*)$, which clarifies the proof of the convergence of the DOBOC-K algorithm.

Lemma 5. *Under Assumptions 1 and 2, if select the step size as*

$$\eta < \min\left\{1, \frac{1}{a}, \frac{2m}{a^2 K^2}\right\}, \tag{29}$$

where $a = M + \frac{2(1-w_{min})}{\lambda}$*, then by DOBOC-K algorithm, there holds*

$$F(x_{k+1}) - F(x_*) \leq \left[1 - \frac{2m^2\eta - ma^2\eta^2K^2}{a}\right](F(x_k) - F(x_*)) + \frac{a^3(2a)^{\frac{3}{2}}\eta^3K^3L}{6m^3}(F(x_k) - F(x_*))^{\frac{3}{2}}. \tag{30}$$

Proof. The DOBOC-K algorithm can be rewritten in a centralized form $x_{k+1} = x_k - h\nabla^2 F(x_k)^{-1}\nabla F(x_k)$, where $h = I - (I - \eta\nabla^2 F(x_k))^K$. Assuming $\eta < \frac{1}{a}$, it can be easily shown that

$$\begin{aligned}
&\lambda_1(h) \leq 1 - (1 - \eta K \lambda_1(\nabla^2 F(x_k))) = \eta K\ \lambda_1(\nabla^2 F(x_k)) \leq a\eta K \\
&\lambda_n(h) \geq 1 - (1 - \eta\lambda_n(\nabla^2 F(x_k))) \geq \eta\ \lambda_n(\nabla^2 F(x_k)) \geq m\eta > 0.
\end{aligned}$$

by using the Bernoulli inequality $(1-x)^k \geq 1 - kx$ for all $x \in (0, 1)$ [27] . It can be verified from

$$\begin{aligned}
&h\nabla^2 F(x)^{-1} \\
=&\nabla^2 F(x)^{-1} - (I - \eta\nabla^2 F(x))(I - \eta\nabla^2 F(x)) \cdots (I - \eta\nabla^2 F(x))\nabla^2 F(x)^{-1} \\
=&\nabla^2 F(x)^{-1} - (I - \eta\nabla^2 F(x))(I - \eta\nabla^2 F(x)) \cdots \nabla^2 F(x)^{-1}(I - \eta\nabla^2 F(x)) \\
&\vdots \\
=&\nabla^2 F(x)^{-1} - \nabla^2 F(x)^{-1}(I - \eta\nabla^2 F(x))(I - \eta\nabla^2 F(x)) \cdots (I - \eta\nabla^2 F(x)) \\
=&\nabla^2 F(x)^{-T} h^T
\end{aligned}$$

and $\lambda_n(h\nabla^2 F(x)^{-1}) > \lambda_n(h)\lambda_n(\nabla^2 F(x)^{-1}) > 0$ that $h\nabla^2 F(x)^{-1} \succ 0$. We can also prove that $\nabla^2 F(x)h \succ 0$ by using the similar method. Substituting y and x with x_{k+1} and x_k respectively in Lemma 4 yields

$$\begin{aligned}
F(x_{k+1}) \leq& F(x_k) + \nabla F(x_k)^T(x_{k+1} - x_k) + \frac{1}{2}(x_{k+1} - x_k)^T\nabla^2 F(x_k)(x_{k+1} - x_k) + \frac{L}{6}||x_{k+1} - x_k||^3 \\
=& F(x_k) - \nabla F(x_k)^T h\nabla^2 F(x_k)^{-1}\nabla F(x_k) + \frac{1}{2}\nabla F(x_k)^T\nabla^2 F(x_k)^{-T} h^T\nabla^2 F(x_k)h\nabla^2 F(x_k)^{-1} \\
&\times \nabla F(x_k) + \frac{L}{6}||h\nabla^2 F(x_k)^{-1}\nabla F(x_k)||^3 \\
\leq& F(x_k) - m\eta\nabla F(x_k)^T\nabla^2 F(x_k)^{-1}\nabla F(x_k) + \frac{a^2\eta^2K^2}{2}\nabla F(x_k)^T\nabla^2 F(x_k)^{-1}\nabla F(x_k) \\
&+ \frac{a^3\eta^3K^3L}{6}||\nabla^2 F(x_k)^{-1}\nabla F(x_k)||^3 \\
\leq& F(x_k) - \frac{2m\eta - a^2\eta^2K^2}{2a}\nabla F(x_k)^T\nabla F(x_k) + \frac{a^3\eta^3K^3L}{6m^3}||\nabla F(x_k)||^3.
\end{aligned} \tag{31}$$

From (25) and (26), one has

$$F(x_*) \geq F(x_k) - \frac{1}{2m}\nabla F(x_k)^T\nabla F(x_k), \tag{32}$$

$$F(x_*) \leq F(x_k) - \frac{1}{2a}\nabla F(x_k)^T\nabla F(x_k). \tag{33}$$

Setting $\eta < \frac{2m}{[a(K-1)]^2}$ and substituting (32) and (33) into (31), one has

$$F(x_{k+1}) - F(x_*) \leq \left[1 - \frac{2m^2\eta - ma^2\eta^2K^2}{a}\right](F(x_k) - F(x_*)) + \frac{a^3(2a)^{\frac{3}{2}}\eta^3K^3L}{6m^3}(F(x_k) - F(x_*))^{\frac{3}{2}}.$$

□

The theorem below shows that the DOBOC-K algorithm can converge to the optimal value $F(x_*)$ with at least linear convergence rate.

Theorem 2. *Consider x_k are generated by the DOBOC-K. If Assumptions 1 and 2 hold, and the step size is chosen to satisfy* (29) *and*

$$\eta < \min\left\{\frac{m}{a^2K^2}, \left[\frac{6m^5}{a^4(2a)^{\frac{2}{3}}K^3L(F(x_0) - F(x_*))^{\frac{1}{2}}}\right]^{\frac{1}{2}}\right\},$$

then the sequence $F(x_k)$ converges to the optimal argument $F(x_*)$ at least linear as

$$F(x_k) - F(x_*) \leq (1-\epsilon)^k(F(x_0) - F(x_*)),$$

where the constant $0 < \epsilon < 1$ is given by

$$\epsilon = \frac{2m^2\eta - ma^2\eta^2K^2}{a} - \frac{a^3(2a)^{\frac{3}{2}}\eta^3K^3L}{6m^3}(F(x_0) - F(x_*))^{\frac{1}{2}}.$$

Proof. From lemma 5, we can rewrite (30) as

$$F(x_{k+1}) - F(x_*) \leq (1-\beta_k)(F(x_k) - F(x_*)),$$

where

$$\beta_k = \frac{2m^2\eta - ma^2\eta^2K^2}{a} - \frac{a^3(2a)^{\frac{3}{2}}\eta^3K^3L}{6m^3}(F(x_k) - F(x_*))^{\frac{1}{2}}. \tag{34}$$

It easily to find

$$\beta_k \leq \frac{2ma\eta K - ma^2\eta^2K^2}{a} \leq \frac{m}{a} < 1.$$

We now turn to proof $\beta_k > 0$. Notice that $2 - \frac{a^2\eta K^2}{m} > 1$ when $\eta < \frac{m}{a^2K^2}$, thus

$$\begin{aligned}\beta_0 =& \frac{2m^2\eta - ma^2\eta^2K^2}{a} - \frac{a^3(2a)^{\frac{3}{2}}\eta^3K^3L}{6m^3}(F(x_0) - F(x_*))^{\frac{1}{2}} \\ \geq& \frac{m^2\eta}{a} - \frac{a^3(2a)^{\frac{3}{2}}\eta^3K^3L}{6m^3}(F(x_0) - F(x_*))^{\frac{1}{2}}.\end{aligned}$$

Set $\eta < \left[\frac{6m^5}{a^4(2a)^{\frac{2}{3}}K^3L(F(x_0)-F(x_*))^{\frac{1}{2}}}\right]^{\frac{1}{2}}$, then $\beta_0 > 0$. It immediately get $F(x_1) - F(x_*) < F(x_0) - F(x_*)$, which implys $\beta_0 < \beta_1$ from (34). Using the similar method, we can prove $0 < \beta_k < \beta_{k+1} < 1$ for all index k. Set $\epsilon = \beta_0$, the proof is completed. □

V. CONCLUSION

In this paper, we proposed a new algorithm called DOBOC to solve distributed optimization problems. The DOBOC algorithm is based on the penalty method interpretation of distributed optimization and optimal control theory. It successfully integrates second-order information into distributed optimization without requiring the inverse of the Hessian matrix. A connection has been established between DOBOC and traditional optimization methods. To balance the trade-off between the number of iterations and communication, we also proposed a variant of the DOBOC, named DOBOC-K. The results on the convergence analysis of DOBOC and DOBOC-K have been achieved, and both algorithms have demonstrated superior performance.

REFERENCES

[1] U. A. Khan, S. Kar, and J. M. Moura, "Diland: An algorithm for distributed sensor localization with noisy distance measurements," *IEEE Transactions on Signal Processing*, vol. 58, no. 3, pp. 1940–1947, 2009.

[2] M. Rabbat and R. Nowak, "Distributed optimization in sensor networks," in *Proceedings of the 3rd international symposium on Information processing in sensor networks*, 2004, pp. 20–27.

[3] I. D. Schizas, A. Ribeiro, and G. B. Giannakis, "Consensus in ad hoc wsns with noisy links—part i: Distributed estimation of deterministic signals," *IEEE Transactions on Signal Processing*, vol. 56, no. 1, pp. 350–364, 2007.

[4] J. F. Mota, J. M. Xavier, P. M. Aguiar, and M. Püschel, "Distributed optimization with local domains: Applications in mpc and network flows," *IEEE Transactions on Automatic Control*, vol. 60, no. 7, pp. 2004–2009, 2014.

[5] Y. Cao, W. Yu, W. Ren, and G. Chen, "An overview of recent progress in the study of distributed multi-agent coordination," *IEEE Transactions on Industrial informatics*, vol. 9, no. 1, pp. 427–438, 2012.

[6] C. G. Lopes and A. H. Sayed, "Diffusion least-mean squares over adaptive networks: Formulation and performance analysis," *IEEE Transactions on Signal Processing*, vol. 56, no. 7, pp. 3122–3136, 2008.

[7] A. Koppel, S. Paternain, C. Richard, and A. Ribeiro, "Decentralized online learning with kernels," *IEEE Transactions on Signal Processing*, vol. 66, no. 12, pp. 3240–3255, 2018.

[8] D. Lee, N. He, P. Kamalaruban, and V. Cevher, "Optimization for reinforcement learning: From a single agent to cooperative agents," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 123–135, 2020.

[9] A. S. Bedi, A. Koppel, and K. Rajawat, "Asynchronous online learning in multi-agent systems with proximity constraints," *IEEE Transactions on Signal and Information Processing over Networks*, vol. 5, no. 3, pp. 479–494, 2019.

[10] J. Zhang, K. You, and T. Başar, "Distributed adaptive newton methods with global superlinear convergence," *Automatica*, vol. 138, p. 110156, 2022.

[11] O. Shamir, N. Srebro, and T. Zhang, "Communication-efficient distributed optimization using an approximate newton-type method," in *International conference on machine learning*. PMLR, 2014, pp. 1000–1008.

[12] S. Wang, F. Roosta, P. Xu, and M. W. Mahoney, "Giant: Globally improved approximate newton method for distributed optimization," *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[13] S. Soori, K. Mishchenko, A. Mokhtari, M. M. Dehnavi, and M. Gurbuzbalaban, "Dave-qn: A distributed averaged quasi-newton method with local superlinear convergence rate," in *International conference on artificial intelligence and statistics*. PMLR, 2020, pp. 1965–1976.

[14] A. Mokhtari, Q. Ling, and A. Ribeiro, "Network newton distributed optimization methods," *IEEE Transactions on Signal Processing*, vol. 65, no. 1, pp. 146–161, 2016.

[15] D. Bajovic, D. Jakovetic, N. Krejic, and N. K. Jerinkic, "Newton-like method with diagonal correction for distributed optimization," *SIAM Journal on Optimization*, vol. 27, no. 2, pp. 1171–1203, 2017.

[16] A. Mokhtari, W. Shi, Q. Ling, and A. Ribeiro, "A decentralized second-order method with exact linear convergence rate for consensus optimization," *IEEE Transactions on Signal and Information Processing over Networks*, vol. 2, no. 4, pp. 507–522, 2016.

[17] M. Eisen, A. Mokhtari, and A. Ribeiro, "A primal-dual quasi-newton method for exact consensus optimization," *IEEE Transactions on Signal Processing*, vol. 67, no. 23, pp. 5983–5997, 2019.

[18] A. Mokhtari, W. Shi, Q. Ling, and A. Ribeiro, "Dqm: Decentralized quadratically approximated alternating direction method of multipliers," *IEEE Transactions on Signal Processing*, vol. 64, no. 19, pp. 5158–5173, 2016.

[19] J. Zhang, Q. Ling, and A. M.-C. So, "A newton tracking algorithm with exact linear convergence for decentralized consensus optimization," *IEEE Transactions on Signal and Information Processing over Networks*, vol. 7, pp. 346–358, 2021.

[20] D. Varagnolo, F. Zanella, A. Cenedese, G. Pillonetto, and L. Schenato, "Newton-raphson consensus for distributed convex optimization," *IEEE Transactions on Automatic Control*, vol. 61, no. 4, pp. 994–1009, 2015.

[21] Y. Xu, Z. Guo, H. Wang, and H. Zhang, "Optimization method based on optimal control," *arXiv preprint arXiv:2309.05280*, 2023.

[22] H. Zhang and H. Wang, "Optimization methods rooting in optimal control," *arXiv preprint arXiv:2312.01334*, 2023.

[23] H. Wang, Y. Xu, Z. Guo, and H. Zhang, "Superlinear optimization algorithms," *arXiv preprint arXiv:2403.11115*, 2024.

[24] D. S. Naidu, *Optimal Control Systems*. Boca Raton: CRC Press, 2018.

[25] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge university press, 2004.

[26] T. M. Apostol, *Calculus, Volume 1*. John Wiley & Sons, 1991.

[27] D. S. Mitrinovic, J. Pecaric, and A. M. Fink, *Classical and new inequalities in analysis*. Springer Science & Business Media, 2013, vol. 61.